ChatGPT

Biometric Gate Access Control System

We propose a secure IoT-based access control system. Leading access-control vendors provide biometric devices and SDKs for integration. For example, Suprema's **BioStar Device SDK** lets developers integrate Suprema fingerprint/face readers with custom software【17†L209-L217】. ZKTeco likewise offers a **ZKFinger SDK** for its fingerprint sensors (on Linux/Windows) to retrieve images/templates 【35†L139-L147】. Using such hardware, we can enroll all 20 residents' fingerprints/faces into the gate device. To handle visitors, residents use a mobile app to pre-authorize guests, issuing a one-time code or QR pass. The gate's touchscreen has a "Visitor/Relative" option: the guest enters the code, which the device (or backend) validates before unlocking. This mirrors modern systems where "visitors are authorized before they arrive" for seamless entry【68†L153-L157】.

Biometric Hardware & SDKs

Hardware: Use a rugged biometric reader at the gate. For example, a Suprema BioStation (fingerprint/face) or BioEntry (face) terminal can be mounted at the gate. These devices come with integrated camera/fingerprint sensors and relays to trigger the gate motor. They support high-accuracy recognition with liveness checks. We would register resident biometrics on each device (or centrally). On power-up, the device holds the enrolled templates and matches scans locally. For integration, vendor SDKs expose functions to enroll users and control the lock. Suprema's BioStar Device SDK (C++/C#) provides a unified API for all Suprema readers【17†L209-L217】. ZKTeco's SDKs (e.g. ZKFinger) similarly allow fingerprint readers to be embedded in custom hardware【35†L139-L147】. These SDKs simplify enrolling templates and receiving access events in our software.

Visitor Pass Input: The gate UI must allow guests to enter a code. When a resident schedules a visit in the app, the backend generates a one-time PIN or QR code tied to that apartment. On arrival, the guest selects "Visitor" and types the PIN on the gate's touchscreen/keypad. The device sends this PIN to the backend (via its network connection) for verification. If valid (and not expired), it opens the gate. This secure code approach (with expiration and one-time use) prevents random guessing. Suprema's platforms even include "visitor credential" workflows to authorize guests in advance【68†L153-L157】. All such codes and visits are logged for audit.

Backend Architecture and Security

Server/API: A secure backend (cloud or on-prem) runs the access-control application. We would build a RESTful API (JSON/HTTPS) for all operations: user management, visitor creation, device events, etc. Best practices must be followed: input validation, parameterized queries, and OWASP API Top 10 mitigations. All endpoints use TLS (HTTPS) and authenticate requests.

Authentication: Use a standard framework like OAuth 2.1 / OpenID Connect. Residents log into the mobile app (and web admin) with username/password (or federation) and receive a short-lived JWT access token【45†L37-L40】. Every API call must present a valid token. The JWT should be signed by our auth server; the API will verify its signature, expiration, and scopes/roles (e.g. `user:john:generate_visitor`). We would use the *Authorization: Bearer* header with TLS for each request. For the gate device itself, which is a fixed client, we would use mutual TLS or a device certificate to authenticate to the API (a service-to-service flow). As one guide notes, "there's no single best method"

for all use cases【45†L37-L40】: OAuth 2.0 (JWT tokens) works well for end-user flows, while mTLS or client-cert auth suits device-to-server links. API keys could be used for admin tools if limited to trusted networks.

Authorization: Inside the API, enforce role-based access control. For instance, a resident can only generate passes for their own apartment, while an admin user can manage all devices. The gate device, after verifying a scan or PIN, only invokes the “open gate” action on its own location.

Data Protection: All sensitive data must be encrypted at rest. The database (e.g. PostgreSQL) should use disk encryption. Biometric templates should never be stored as plain images or identities; either keep them only on the secure reader hardware, or if stored centrally, store as encrypted templates or secure hashes. Visitor records (names, codes) should also be encrypted. Logs of access attempts are critical; as recommended, we should “encrypt visitor data and automatically save audit logs” so that access events are tamper-evident【68†L159-L164】. Audit logs (including failed attempts) should be append-only and periodically backed up to a secure archive.

Mobile App and Notifications

App Framework: A cross-platform framework (Flutter or React Native) is ideal for the resident mobile app. For example, Google’s Flutter (Dart) or Facebook’s React Native (JavaScript) let us write one codebase for iOS and Android【57†L94-L102】【57†L104-L112】. Flutter is a portable UI toolkit for high-performance native apps【57†L94-L102】, while React Native (used by Instagram, Tesla, etc.) builds iOS/Android apps with JavaScript【57†L104-L112】. The app’s functions: resident login, list family members, create a new visitor pass (select contact, set time window), and display active passes. It communicates with our backend over HTTPS and authenticates via OAuth/JWT.

Push Notifications: To alert residents in real time, use a push service like Firebase Cloud Messaging (FCM). FCM is a cross-platform messaging solution for iOS/Android【58†L1-L4】. When a visitor code is used at the gate, the server can send a push notification (via FCM) to the resident’s phone: e.g. “Guest Alice has arrived at 3:47 PM.” Setting up FCM in Flutter or React Native is straightforward using the Firebase SDK. This ensures residents instantly know when someone uses the gate. (On iOS, FCM will route through Apple’s APNs under the hood.)

System Architecture

In summary, the system comprises:

- **Gate IoT Device:** Runs the biometric scanner and gate control. This device may be a small PC (Windows/Linux) or a specialized embedded system. It interfaces with the biometric sensor hardware (via the vendor’s SDK or API) and with the gate motor (via a relay or I/O signal). It maintains a network connection (Ethernet/Wi-Fi/4G) and periodically syncs with the server for updates (user enrollments, visitor codes). Upon a scan or code entry, it calls the server API to log the attempt and verify authorizations.

- **Network:** The gate device should be on a secured network segment (e.g. separate VLAN). Firewall rules restrict outbound traffic only to the backend server’s API endpoint (and DNS). All communications use TLS 1.2+.

- **Backend Cloud:** We recommend deploying the server in a cloud environment (AWS/GCP/Azure) for scalability and reliability. For example, use a managed Kubernetes cluster or container service to host microservices (Auth service, User/Visitor service, Device service). An API Gateway (or NGINX) routes requests and handles SSL termination. The database (e.g. AWS RDS) stores the application data. Use multiple availability zones for high availability.

- **Auth Service:** A component (or third-party like AWS Cognito/Auth0) handles resident login, token issuance, and stores user credentials (hashed).

- **Web Admin Panel:** A web interface for property managers, built on the same backend, allows viewing logs, adding devices, emergency override, and system settings.

- **Reliability:** To avoid a single point of failure, run multiple instances behind load balancers. Use health checks and auto-scaling. Regularly back up the database.

This multi-tier architecture (device → API → app/db) is similar to other cloud-based IoT solutions: Suprema, for instance, offers both an on-prem access-control platform (**BioStar X**) and a cloud platform (**BioStar Air**) for managing devices and users【12†L244-L252】【12†L262-L270】. We could adopt a similar model: e.g. run our own cloud server (like BioStar X) or use a hosted service (akin to BioStar Air) to avoid managing servers. In either case, encryption and token-based auth are mandatory.

Deployment and Installation Steps

1. **Develop & Test Software:** Build the backend services (user, device, visitor, log services) using a secure framework (e.g. Node.js/Express or Python/Django). Develop the gate device app using the vendor's SDK (e.g. in C# or C++ on Windows, or Python on Linux). Build the mobile app with Flutter or React Native. Test end-to-end using simulators or dev boards.

2. **Set Up Servers:** Provision cloud servers. Configure environment (e.g. Docker images, Kubernetes). Install TLS certificates (via Let's Encrypt or a CA) for all domains. Implement CI/CD pipelines for automated deployment.

3. **Database & Storage:** Initialize the SQL database with encrypted storage. Run migrations for user/visitor tables. Set up a backup schedule.

4. **Configure Gate Device:** At the apartment gate, mount the reader and connect to power and to the gate motor control. Configure network (Ethernet preferred). Install and configure the IoT device software, pointing it to the backend endpoint. Install the biometric SDK drivers. Change all factory passwords. Test device connectivity.

5. **Enroll Residents:** Either via the app or a local admin mode, capture each resident's fingerprint/face and register it with the system. Ensure the device has copies of all enrolled templates (or test recognition via server if using cloud matching).

6. **Visitor Flow:** Test visitor pass creation. As a resident, use the app to create a visitor with a PIN. At the gate, press "Visitor," enter the PIN, and verify the gate opens. Check that both success and failure are logged.

7. **Monitoring:** Set up logging/monitoring (CloudWatch, ELK stack, or Grafana) to track system health and alerts. Monitor device heartbeats (the gate device can ping the server periodically).

8. **Documentation:** Provide manuals for onsite staff: how to reset the system, override access in emergencies, and update firmware. Train residents on using the app.

Security Hardening

- **Network Security:** Isolate IoT devices on their own VLAN. Use strict firewall rules – only allow outbound traffic to the server's IP and block all inbound. If using Wi-Fi, use WPA2/WPA3 with a strong password and disable guest networks near the gate. Prefer a wired connection or a secure LTE router.

- **Encryption:** Enforce HTTPS/TLS for all API traffic. Use strong cipher suites. Store database and backups encrypted. Use hardware-backed keys if available. On the device, enable disk encryption and protect the OS bootloader.

- **Authentication & Secrets:** Do not hard-code secrets. Store certificates/keys securely (e.g. AWS Secrets Manager, Azure Key Vault). Rotate API keys and credentials periodically. Use OAuth scopes to limit token permissions.

- **Firmware & Updates:** Keep device firmware and software up-to-date. Disable unused services (e.g. SSH) on the IoT device. Enable secure boot if supported. All SDK and OS components should be on the latest stable versions to patch vulnerabilities.

- **Input Validation:** On the backend, rigorously validate all inputs (visitor names, PIN codes, timestamps). Sanitize strings to prevent injection. Implement rate limiting on login/PIN entry endpoints to prevent brute force. For the gate UI, disable arbitrary code execution – only allow the defined menu actions.

- **Audit & Monitoring:** Log all access attempts (success and failure), API calls, and admin actions. Store logs centrally and monitor for anomalies (e.g. repeated failed codes). As recommended for physical access systems, encryption and auditability are key to compliance【68†L159-L164】. Regularly review logs and conduct security drills (e.g. simulate an attack).

- **Failover Plans:** Design a safe failure mode. For example, if the server is unreachable, the gate device could default to allowing only resident biometrics (it has them cached) but block new visitor codes. Also have an emergency manual key/override for authorized personnel in case of system failure.

- **Privacy:** Handle biometric data with care. The system should comply with relevant privacy laws (e.g. GDPR) by minimizing data retention and securing user consent. Biometric templates should never be exposed or transmitted in clear; best practice is to keep them only in the secure module of the reader.

By leveraging proven biometric hardware and cloud technologies, and by following stringent API and IoT security practices【72†L106-L109】【72†L207-L213】, this architecture will provide robust, future-proof access control for the apartment. The combination of on-site biometric gates, secure mobile apps, and hardened backend servers ensures that only authorized individuals can enter, with full audit trails and encryption guarding every step of the process.

Sources: Suprema BioStar SDK documentation【17†L209-L217】, ZKTeco developer guides【35†L139-L147】, industry best practices for OAuth/IoT security【45†L37-L40】【72†L106-L109】, and access-

control design patterns【68†L153-L157】【68†L159-L164】. These informed the above tech stack and design.